

BookWorm Engineer API Map and Expansion Guide

Purpose

This document explains the APIs used inside the BookWorm Engineer project and how teammates should extend them safely.

The project follows one main architecture rule:

Imports should be boring, execution should be explicit.

That means importing a module should not start the app, load `.env`, create clients, read runtime environment variables, or begin the agent loop.

Runtime startup should happen only through:

```
bookworm
```

or:

```
python -m bookworm
```

High-Level Architecture

The current architecture is:

```
cli.py
  loads .env, creates config/client/tools/prompt

config.py
  owns environment parsing and validation

llm.py
  owns model provider setup

prompts.py
  owns system prompt construction

agent.py
  owns conversation loop and tool-call orchestration
```

```
tools/  
  owns safe capabilities exposed to the model
```

The intended dependency direction is:

```
cli.py  
  -> config.py  
  -> llm.py  
  -> prompts.py  
  -> tools/registry.py  
  -> agent.py
```

`agent.py` should receive dependencies. It should not create global runtime dependencies during import.

Quick Reference Table

Area	File	Main API	Purpose
CLI startup	<code>bookworm/cli.py</code>	<code>main()</code>	Starts the app and wires dependencies
Config	<code>bookworm/config.py</code>	<code>load_config()</code> / <code>Config</code>	Reads and validates runtime settings
LLM client	<code>bookworm/llm.py</code>	<code>create_client()</code>	Creates OpenAI-compatible client
Prompt	<code>bookworm/prompts.py</code>	<code>build_system_prompt()</code>	Builds system prompt from config and project files
Agent runtime	<code>bookworm/agent.py</code>	<code>Agent.run()</code>	Runs conversation loop and tool calls
Tool registry	<code>bookworm/tools/registry.py</code>	<code>create_tool_registry()</code>	Binds schemas to implementations
Tool schemas	<code>bookworm/tools/schema.py</code>	<code>SCHEMA</code>	Defines tools visible to the LLM
Tool implementations	<code>bookworm/tools/implementation.py</code>	<code>read_file</code> , <code>write_file</code> , <code>bash</code> , <code>ask_user</code>	Implements local capabilities
Console command	<code>pyproject.toml</code>	<code>bookworm = "bookworm.cli:main"</code>	Allows running <code>bookworm</code>

Area	File	Main API	Purpose
Module command	bookworm/ __main__.py	main()	Allows running python -m bookworm

External APIs

OpenAI SDK

Used in:

```
bookworm/llm.py
bookworm/agent.py
```

Main calls:

```
OpenAI(api_key=..., base_url=...)
client.chat.completions.create(...)
```

Purpose

The OpenAI SDK is used to:

- create an OpenAI-compatible client
- call OpenRouter or OpenAI chat completions
- pass tool schemas to the model
- receive assistant messages
- receive tool calls from the model

Current Usage

Client creation should happen in:

```
bookworm/llm.py
```

Example:

```
from openai import OpenAI

from .config import Config
```

```
def create_client(config: Config) -> OpenAI:
    return OpenAI(
        api_key=config.llm_api_key,
        base_url=config.llm_base_url,
        default_headers={
            "X-OpenRouter-Title": "BookWorm Engineer",
        },
    )
```

Model calls should happen inside:

```
bookworm/agent.py
```

Example shape:

```
response = self.client.chat.completions.create(
    model=self.config.llm_model,
    messages=self.messages,
    tools=self.tool_registry.schema,
)
```

Scaling Guidance

Keep provider-specific setup in `llm.py`.

Good things to add to `llm.py` later:

- provider switching
- OpenRouter-specific headers
- default model options
- retry policy
- request timeout settings
- logging hooks
- fallback providers
- mock clients for tests

Avoid adding provider setup directly to `agent.py`.

`python-dotenv`

Used in:

```
bookworm/cli.py
```

Main call:

```
load_dotenv(repo_root / ".env")
```

Purpose

`python-dotenv` loads local environment variables before config validation.

The `.env` file is useful for local development because teammates can define values like:

```
LLM_API_KEY=your_api_key_here
LLM_BASE_URL=https://openrouter.ai/api/v1
LLM_MODEL=openai/gpt-4o-mini
CONTEXT_WINDOW=128000
GIT_USER_NAME=Your Name
GIT_USER_EMAIL=you@example.com
```

Scaling Guidance

Keep `.env` loading only in `cli.py`.

Do not load `.env` from:

```
__init__.py
agent.py
config.py
llm.py
tools/
```

This keeps imports safe and predictable.

subprocess

Used in:

```
bookworm/tools/implementation.py
```

Main call:

```
subprocess.run(...)
```

Purpose

`subprocess` allows the agent to run allowlisted terminal commands through the `bash` tool.

This is one of the highest-risk parts of the project because it gives the agent access to the local shell.

Scaling Guidance

Keep all shell behavior centralized in `implementation.py` for now.

Important safety controls should stay together:

- command allowlisting
- command denylisting
- timeout
- working directory locking
- minimal environment handling
- stdout/stderr capture
- clear error reporting

Eventually, when the file grows, split shell logic into:

```
bookworm/tools/shell.py
```

Do not scatter shell execution across the codebase.

Internal APIs

`Config` and `load_config`

Defined in:

```
bookworm/config.py
```

Main usage:

```
config = load_config(working_dir=Path.cwd())
```

Purpose

`Config` centralizes runtime settings.

It should own:

- `LLM_API_KEY`
- `LLM_BASE_URL`
- `LLM_MODEL`
- `CONTEXT_WINDOW`

- `GIT_USER_NAME`
- `GIT_USER_EMAIL`
- `WORKING_DIR`

The rest of the application should use the `Config` object instead of reading environment variables directly.

Example Shape

```
from dataclasses import dataclass
from pathlib import Path

@dataclass(frozen=True)
class Config:
    llm_api_key: str
    llm_base_url: str
    llm_model: str
    context_window: int
    git_user_name: str
    git_user_email: str
    working_dir: Path
```

Scaling Guidance

When adding a new runtime setting:

1. Add it to `.env.example`.
2. Add it to `Config`.
3. Parse and validate it in `load_config`.
4. Pass `Config` into the module that needs it.
5. Access it through `self.config` or a function argument.

Do not do this inside random modules:

```
os.environ["NEW_SETTING"]
```

Prefer this:

```
config.new_setting
```

`create_client`

Defined in:

```
bookworm/llm.py
```

Main usage:

```
client = create_client(config)
```

Purpose

`create_client` builds the OpenAI-compatible client from `Config`.

This keeps model-provider details outside of the agent loop.

Scaling Guidance

Use `llm.py` as the provider boundary.

Future changes that belong here:

- switching between OpenAI and OpenRouter
- adding Anthropic or other providers through adapters
- default request headers
- timeout configuration
- retry configuration
- client instrumentation
- test doubles or fake clients

`build_system_prompt`

Defined in:

```
bookworm/prompts.py
```

Main usage:

```
system_prompt = build_system_prompt(config)
```

Purpose

`build_system_prompt` creates the system prompt used by the agent.

It may include:

- static system instructions

- `config.working_dir`
- Git identity information
- `AGENTS.md`
- `PROGRESS.md`

Scaling Guidance

Prompt construction should stay in `prompts.py`.

Do not put large prompt strings back into:

```
agent.py  
cli.py  
llm.py
```

If the prompt needs more project context later, add that logic to `prompts.py`.

Good future additions:

- reading `.bookworm/instructions.md`
- reading project summaries
- adding configurable prompt sections
- supporting prompt templates
- supporting different prompt modes

Agent

Defined in:

```
bookworm/agent.py
```

Main usage:

```
agent = Agent(  
    config=config,  
    client=create_client(config),  
    tool_registry=create_tool_registry(config),  
    system_prompt=build_system_prompt(config),  
)  
  
agent.run()
```

Purpose

`Agent` owns the runtime conversation loop.

It should handle:

- storing message history
- accepting user input
- sending messages to the model
- passing tool schemas to the model
- receiving assistant responses
- detecting tool calls
- executing tool calls through the tool registry
- appending tool results back into message history
- tracking context usage
- printing final assistant replies

Scaling Guidance

`agent.py` should focus on orchestration, not setup.

Do not add these to module-level code in `agent.py`:

```
os.environ["LLM_API_KEY"]
OpenAI(...)
load_dotenv(...)
create_tool_registry(...)
build_system_prompt(...)
```

Instead, receive dependencies in the constructor:

```
class Agent:
    def __init__(
        self,
        config: Config,
        client: OpenAI,
        tool_registry: ToolRegistry,
        system_prompt: str,
    ) -> None:
        ...
```

This keeps the agent testable and reusable.

Tool APIs

The tools are the capabilities exposed to the model.

Current tools:

```
read_file
write_file
bash
ask_user
```

The tool system has three parts:

```
schema.py
    describes tools to the model

implementation.py
    contains the Python functions

registry.py
    binds schema names to callable implementations
```

ToolRegistry and create_tool_registry

Defined in:

```
bookworm/tools/registry.py
```

Main usage:

```
tool_registry = create_tool_registry(config)
```

Purpose

The tool registry connects tool schemas to actual Python functions.

It allows `agent.py` to stay generic.

Instead of hardcoding tool functions directly in the agent loop, the agent can call:

```
self.tool_registry.implementations[fn_name](**fn_args)
```

Expected Shape

```
@dataclass
class ToolRegistry:
```

```
schema: list[dict]
implementations: dict[str, Callable[..., str]]
```

Example registration:

```
return ToolRegistry(
    schema=SCHEMA,
    implementations={
        "read_file": read_file,
        "write_file": write_file,
        "bash": bash,
        "ask_user": ask_user,
    },
)
```

Scaling Guidance

Every new tool should be added in three places:

1. Add the tool schema in `schema.py`.
2. Add the function implementation in `implementation.py`.
3. Register the schema name to the function in `registry.py`.

The tool name in the schema must match the key in `implementations`.

Example:

```
# schema.py
{
    "type": "function",
    "function": {
        "name": "read_file",
        ...
    },
}
```

Must match:

```
# registry.py
"read_file": read_file
```

Tool Schemas

Defined in:

bookworm/tools/schema.py

Current export:

SCHEMA

Purpose

Tool schemas tell the model:

- what tools exist
- what each tool does
- what arguments each tool accepts
- which arguments are required

Example Schema

```
{
  "type": "function",
  "function": {
    "name": "read_file",
    "description": "Read a file from the working directory.",
    "parameters": {
      "type": "object",
      "properties": {
        "file_path": {
          "type": "string",
          "description": "Path to the file relative to the working
directory.",
        }
      },
      "required": ["file_path"],
      "additionalProperties": False,
    },
  },
}
```

Scaling Guidance

Keep schemas explicit and boring.

Recommended schema rules:

- Use clear descriptions.
- Use `additionalProperties: False` where possible.
- Keep argument names stable.

- Make required arguments explicit.
- Avoid overly broad tools.
- Avoid tools that can perform many unrelated actions.

Good tool:

```
read_file
```

Risky tool:

```
do_anything_on_computer
```

Tool Implementations

Defined in:

```
bookworm/tools/implementation.py
```

Current callable APIs:

```
read_file(file_path: str) -> str
write_file(file_path: str, content: str) -> str
bash(command: str) -> str
ask_user(question: str) -> str
```

Purpose

Tool implementations are the actual local capabilities the model can call.

They should return strings because tool results are passed back into the model conversation.

Current Tools

```
read_file
```

```
read_file(file_path: str) -> str
```

Reads a file inside the configured working directory.

Expected behavior:

- resolve the path safely

- prevent path traversal outside the working directory
 - return file content
 - return a clear error string if the file cannot be read
-

`write_file`

```
write_file(file_path: str, content: str) -> str
```

Writes content to a file inside the configured working directory.

Expected behavior:

- resolve the path safely
 - prevent writing outside the working directory
 - create parent folders if needed
 - write UTF-8 text
 - return a success or error string
-

`bash`

```
bash(command: str) -> str
```

Runs an allowlisted shell command inside the configured working directory.

Expected behavior:

- validate command against policy
 - block dangerous commands
 - run with a timeout
 - capture stdout
 - capture stderr
 - return the exit code
-

`ask_user`

```
ask_user(question: str) -> str
```

Asks the user for more information during an agent run.

Expected behavior:

- display the question clearly
- return the user's answer as a string

Scaling Guidance

Keep implementations narrow.

A tool should do one clear thing.

If `implementation.py` becomes too large, split it into:

```
bookworm/tools/filesystem.py
bookworm/tools/shell.py
bookworm/tools/user.py
```

Suggested future split:

```
filesystem.py
    read_file
    write_file
    path validation helpers

shell.py
    bash
    allowlist
    denylist
    timeout
    subprocess handling

user.py
    ask_user

registry.py
    imports implementations and registers tools
```

CLI APIs

Console Command

Defined in:

```
pyproject.toml
```

Current entry:


```
[project.scripts]
bookworm = "bookworm.cli:main"
```

Purpose

This allows the user to run:

```
bookworm
```

Setup Requirement

The console command works after installing the package in editable mode:

```
pip install -e .
```

Use this during development so changes to source files are immediately reflected.

Module Entry Point

Defined in:

```
bookworm/__main__.py
```

Main command:

```
python -m bookworm
```

Purpose

This is a fallback entry point that does not rely on the console script being installed.

Expected file:

```
from .cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

Both entry points should go through the same startup path:

```
bookworm.cli:main
```

Adding a New API or Feature

Use this checklist when expanding the project.

Adding a New Environment Setting

Example: adding `MAX_TOOL_CALLS`.

Steps:

1. Add it to `.env.example`.

```
MAX_TOOL_CALLS=20
```

1. Add it to `Config`.

```
max_tool_calls: int
```

1. Parse it in `load_config`.

```
max_tool_calls=_int_env("MAX_TOOL_CALLS", 20)
```

1. Use it through `config`.

```
self.config.max_tool_calls
```

Do not read it directly from `os.environ` in `agent.py`.

Adding a New Tool

Example: adding `list_files`.

Step 1: Add schema in `schema.py`

```
{
    "type": "function",
    "function": {
        "name": "list_files",
        "description": "List files inside a directory under the working
```

```

directory.",
    "parameters": {
        "type": "object",
        "properties": {
            "directory": {
                "type": "string",
                "description": "Directory path relative to the working
directory.",
            }
        },
        "required": ["directory"],
        "additionalProperties": False,
    },
}

```

Step 2: Add implementation in `implementation.py`

```

def list_files(directory: str) -> str:
    ...

```

Step 3: Register it in `registry.py`

```

implementations={
    "read_file": read_file,
    "write_file": write_file,
    "bash": bash,
    "ask_user": ask_user,
    "list_files": list_files,
}

```

Step 4: Test that schema name and implementation key match

The schema name:

```
"name": "list_files"
```

must match:

```
"list_files": list_files
```

Adding a New LLM Provider

Add provider-specific logic to:

```
bookworm/llm.py
```

Not to:

```
bookworm/agent.py
```

Recommended approach:

```
def create_client(config: Config) -> OpenAI:
    ...
```

Later, this can evolve into:

```
def create_client(config: Config) -> LLMClientProtocol:
    ...
```

or:

```
def create_provider(config: Config) -> BaseProvider:
    ...
```

Keep the agent dependent on behavior, not provider-specific setup.

Adding More Prompt Context

Add prompt construction logic to:

```
bookworm/prompts.py
```

Good examples:

- load project-specific instructions
- load summaries
- load `.bookworm` memory
- add repo metadata
- add selected source documents

Avoid adding large prompt blocks directly into `agent.py`.

Safety Rules for Future Expansion

1. Do not start runtime behavior during import

Avoid this:

```
client = OpenAI(...)
agent = Agent(...)
agent.run()
```

at module level.

Prefer this:

```
def main() -> int:
    ...
```

2. Do not read environment variables everywhere

Avoid this:

```
model = os.environ["LLM_MODEL"]
```

Prefer this:

```
model = config.llm_model
```

3. Keep shell execution centralized

Avoid adding `subprocess.run` in random modules.

Keep shell access in:

```
bookworm/tools/implementation.py
```

or later:

```
bookworm/tools/shell.py
```

4. Keep tools explicit

Each tool should have:

```
schema
implementation
registry entry
```

Avoid tools with vague powers.

5. Keep `agent.py` focused on orchestration

`agent.py` should coordinate the conversation loop.

It should not own:

- `.env` loading
 - config parsing
 - prompt construction
 - client construction
 - raw tool implementation logic
-

Recommended File Ownership

`bookworm/cli.py`

Owned by startup and packaging maintainers.

Change this when:

- adding a new app startup dependency
 - changing how `.env` is loaded
 - changing CLI entry behavior
 - adding flags or command-line arguments later
-

`bookworm/config.py`

Owned by configuration maintainers.

Change this when:

- adding env vars

- validating settings
 - changing defaults
 - adding config-derived values
-

`bookworm/llm.py`

Owned by LLM/provider maintainers.

Change this when:

- changing OpenRouter/OpenAI setup
 - adding provider headers
 - changing model defaults
 - adding retry behavior
 - adding provider switching
-

`bookworm/prompts.py`

Owned by prompt/application behavior maintainers.

Change this when:

- changing system prompt text
 - adding `AGENTS.md` or `PROGRESS.md` handling
 - adding project memory
 - changing instruction formatting
-

`bookworm/agent.py`

Owned by runtime orchestration maintainers.

Change this when:

- changing the conversation loop
 - changing tool-call handling
 - changing context usage tracking
 - changing model response handling
 - changing how assistant output is displayed
-

`bookworm/tools/schema.py`

Owned by tool interface maintainers.

Change this when:

- adding a new tool
 - changing tool arguments
 - changing tool descriptions
-

`bookworm/tools/implementation.py`

Owned by tool behavior maintainers.

Change this when:

- changing actual local tool behavior
 - changing file handling
 - changing shell execution
 - changing user input handling
-

`bookworm/tools/registry.py`

Owned by tool integration maintainers.

Change this when:

- adding a new implementation to the registry
 - renaming a tool
 - changing how tools receive config
-

Common Mistakes to Avoid

Mistake 1: Adding env reads inside `agent.py`

Bad:

```
LLM_MODEL = os.environ["LLM_MODEL"]
```

Good:

```
self.config.llm_model
```

Mistake 2: Adding new tools only to `implementation.py`

Adding a function is not enough.

A complete tool needs:

```
schema.py
implementation.py
registry.py
```

Mistake 3: Putting prompt text back into `agent.py`

Bad:

```
SYSTEM_PROMPT = """very long prompt..."""
```

Good:

```
system_prompt = build_system_prompt(config)
```

Mistake 4: Creating the client inside `agent.py`

Bad:

```
client = OpenAI(...)
```

Good:

```
client=create_client(config)
```

Mistake 5: Forgetting editable install for the CLI command

If this does not work:

```
bookworm
```

Run:

```
pip install -e .
```

Or use:

```
python -m bookworm
```

Expansion Decision Guide

Use this guide when deciding where new code belongs.

"I need a new environment variable."

Add it to:

```
config.py  
.env.example
```

Use it through:

```
config.some_value
```

"I need to change the model provider."

Add it to:

```
llm.py
```

"I need to change the system prompt."

Add it to:

```
prompts.py
```

“I need to change how the agent loops.”

Add it to:

```
agent.py
```

“I need to add a new tool.”

Add it to:

```
tools/schema.py  
tools/implementation.py  
tools/registry.py
```

“I need to change how the app starts.”

Add it to:

```
cli.py  
__main__.py  
pyproject.toml
```

Final Summary

The project is organized around explicit startup and boring imports.

The clean flow is:

```
bookworm command  
-> bookworm.cli:main  
-> load .env  
-> load Config  
-> create LLM client  
-> create ToolRegistry  
-> build system prompt  
-> create Agent  
-> run Agent
```

The most important rule for future work is:

Add capabilities by passing dependencies through `Config`, `ToolRegistry`, or constructor arguments. Do not add hidden startup behavior during import.

Following this keeps the project easy to test, easy to package, and safe to extend.